

CSCI3340 Scientific Computing

2025–26 Term 2

Assignment 2

Submission Deadline: 23:59:00, 19th March 2026.

Submission Guidelines

Assignment 2 covers material from Lectures 4–6 and is worth a total of 100 points. There are 6 problems in total: the first 3 are written questions, and the remaining 3 are programming tasks.

You should submit a single .zip file containing exactly one PDF file and all relevant MATLAB files. The PDF must include your solutions to all problems and should be self-contained, clear, and easy to read. You may write your answers by hand or type them. For each programming problem, clearly indicate in your PDF which MATLAB file corresponds to that problem. Your PDF should also include brief instructions on how to run your code and interpret the results.

If the submission was late but within 3 days after the deadline, a -30 points penalty will be applied. Submissions made after this period will not be graded.

If you have any concerns, feel free to contact me (colintsang@cuhk.edu.hk) or the TA in charge (srijan25@link.cuhk.edu.hk).

Problem 1 (10 points)

- (a) Consider an interpolation problem. Let $x_0 < x_1 < \dots < x_n$ be the $n+1$ distinct nodes such that $P_n(x)$ is the unique interpolation polynomial satisfying $P_n(x_i) = f(x_i)$ for all $i = 0, \dots, n$. Assume that f is sufficiently differentiable in the interval $[x_0, x_n]$. Prove that the error at $x \in [x_0, x_n]$ is given by

$$R(x) := f(x) - P_n(x) = \frac{L(x)}{(n+1)!} f^{(n+1)}(\delta)$$

where $\delta \in [x_0, x_n]$ and $L(x) := \prod_{i=0}^n (x - x_i)$.

- (b) Consider $f(x) = x^4$ on $[0, 1]$ with $n = 2$.
- (i) Find an upper bound for $|f'''|$ on the interval $[0, 1]$.
 - (ii) Find an upper bound for $L(x)$ on the interval $[0, 1]$.
 - (iii) Find an upper bound for $R(x)$,

Problem 2 (20 points)

Consider a least square problem $Ax = b$, where $A \in \mathbb{R}^{m \times n}$ is a tall matrix with $m > n$.

- (a) In the lecture, we discussed using the Normal Equations of Gauss, $A^T Ax = A^T bx$, to find the solution of the least square problem. Explain one condition where the Normal Equations of Gauss will fail.
- (b) Suppose we already have the eigendecomposition $A^T A = Q\Lambda Q^T$, where Q is orthogonal and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ contains the eigenvalues of $A^T A$ in the diagonal entries. Write down a method, with explanation, to compute the SVD decomposition of A .
- (c) Consider $A = \begin{pmatrix} 1 & 0 \\ 0 & 4 \\ 0 & 0 \end{pmatrix}$ and $b = \begin{pmatrix} 3 \\ 8 \\ 7 \end{pmatrix}$. Find the least squares solution using the pseudoinverse A^+ .

Problem 3 (10 points)

Let A be a diagonalizable matrix with eigenvalues $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n| > 0$ and normalized eigenvectors $v_1, \dots, v_n$. Also, consider an initial vector $x_0 = \sum_j a_j v_j$ such that $a_n \neq 0$.

- (a) Write down an iterative formula for $\{x_k\}$ such that the sequence approach the direction of v_n , which corresponds to the smallest eigenvalue $|\lambda_n|$. You should avoid the direct computation of A^{-1} .
- (b) Prove the convergence of your iterative formula in part (a).

Problem 4 (20 points)

Let n be even and define the equispaced nodes

$$x_j = \frac{2\pi j}{n}, \quad j = 0, 1, \dots, n-1.$$

Given samples $y_j = f(x_j)$, define the DFT coefficients

$$c_k = \frac{1}{n} \sum_{j=0}^{n-1} y_j e^{-2\pi i j k / n}, \quad k = 0, 1, \dots, n-1.$$

The trigonometric interpolant is

$$p_n(x) = \sum_{k=-n/2}^{n/2} 'c_k e^{ikx},$$

where the $k = \pm n/2$ terms have weight $\frac{1}{2}$.

- (a) Write `c_dft = dft_direct(y)` that computes c_k using the definition above (do not use `fft`). State the cost in big- O .

(b) Write `p = triginterp_eval(c,xq)` to evaluate $p_n(x)$ at query points `xq`. Make sure your code handles the index shift from $k = 0, \dots, n-1$ to $k = -n/2, \dots, n/2$ and the $\frac{1}{2}$ end weights.

(c) For $f(x) = e^{\sin x}$ and $n = 16$, compute $y_j = f(x_j)$, form c , and verify

$$\max_{0 \leq j \leq n-1} |p_n(x_j) - y_j| < 10^{-10}.$$

Report the max error.

(d) Write `c_fft = dft_fft(y)` using MATLAB `fft` with correct scaling. For $n = 2^m$ with $m = 4, 6, 8, 10, 12$, compare `dft_direct` and `dft_fft` by relative $\|\cdot\|_\infty$ error and runtime (use `rng(42)`).

Problem 5 (20 points)

Given the data:

w	48	61	81	113	131
p	91	98	103	110	112

Assume the model

$$p \approx \beta_0 + \beta_1 \ln(w).$$

Let $\beta = [\beta_0, \beta_1]^T$. Use least squares via an SVD-based pseudoinverse. **Do not use `pinv`.**

(a) Form A and b so that $A\beta \approx b$. Write A and b explicitly from the data (no rounding of $\ln(w)$), and state the size of A .

(b) Construct

$$A^+ = VS^{-1}U^T$$

(by inverting the singular values in S), then compute

$$\hat{\beta} = A^+b.$$

Report $\hat{\beta}_0, \hat{\beta}_1$ (6 s.f.) and the singular values.

(c) Compute $r = b - A\hat{\beta}$ and report $\|r\|_2$ and $\|r\|_2/\|b\|_2$. Make one figure: data points (w_i, p_i) and the fitted curve

$$\hat{p}(w) = \hat{\beta}_0 + \hat{\beta}_1 \ln(w) \quad \text{for } w \in [40, 150].$$

(d) Compute

$$\kappa_2(A) = \frac{\sigma_1}{\sigma_2}.$$

With $\tilde{b} = b + 0.1 \text{randn}(\text{size}(b))$, compute $\tilde{\beta} = A^+\tilde{b}$ and report

$$\frac{\|\tilde{\beta} - \hat{\beta}\|_2}{\|\hat{\beta}\|_2}.$$

Problem 6 (20 points)

Imagine you are an engineer at a small search engine tasked with ranking 20 web pages. You model the web as a directed graph, where each page distributes its “influence” equally among the pages it links to. This produces a **20×20 column-stochastic transition matrix**, every column sums to 1, which you can simulate in MATLAB as:

```
A = rand(20);  
A = A ./ sum(A);
```

The ranking of each page is determined by the **eigenvector corresponding to the dominant eigenvalue** of A . By the Perron–Frobenius theorem, this dominant eigenvalue is always exactly 1 for a column-stochastic matrix. But computing the full eigenvalue spectrum of A tells you much more: the **gap between the first and second largest eigenvalues** controls how quickly the ranking converges, and the remaining eigenvalues reveal the deeper spectral structure of your web graph.

You decide to compute all eigenvalues using the **Shifted QR iteration**, which deflates the matrix one eigenvalue at a time. **Implement** `myShiftedQR(A, maxIter, tol)` using the Rayleigh quotient shift. At each step:

- i. Use the bottom-right entry $A_k(n, n)$ as the shift μ_k .
- ii. Factor $A_k - \mu_k I = Q_k R_k$.
- iii. Update $A_{k+1} = R_k Q_k + \mu_k I$.

However, a colleague points out that since you only *care* about the dominant eigenvalue for ranking purposes, computing all eigenvalues is overkill. They suggest **power iteration** instead which is a far simpler method that repeatedly multiplies by A and renormalizes. **Implement** `myPowerIteration(A, tol)` to compute only this dominant eigenvalue.

Now put both methods head-to-head:

- (a) As mentioned above, implement the required functions and run both methods on matrix A and verify your computed dominant eigenvalue against MATLAB's `eig(A)`. You are allowed to use the MATLAB's built-in function `qr(A)`.
- (b) On a single **semilogy** plot, show the eigenvalue error vs. iteration number for both methods.
- (c) **Discussion:** Given that the dominant eigenvalue is known to be exactly 1, does the Rayleigh quotient shift in your QR method help or hinder convergence on this particular eigenvalue? Which method would you recommend to your colleague for the web ranking task? Justify your answer using both your plot and theory.

Problem 1 (10 points)

- (a) Consider an interpolation problem. Let $x_0 < x_1 < \dots < x_n$ be the $n+1$ distinct nodes such that $P_n(x)$ is the unique interpolation polynomial satisfying $P_n(x_i) = f(x_i)$ for all $i = 0, \dots, n$. Assume that f is sufficiently differentiable in the interval $[x_0, x_n]$. Prove that the error at $x \in [x_0, x_n]$ is given by

$$R(x) := f(x) - P_n(x) = \frac{L(x)}{(n+1)!} f^{(n+1)}(\delta)$$

where $\delta \in [x_0, x_n]$ and $L(x) := \prod_{i=0}^n (x - x_i)$.

(1) When $x = x_0, x_1, \dots, x_n$, $R(x) = 0 = f(x) - P_n(x)$.

(2) Consider fixed point $x \neq x_0, x_1, \dots, x_n$

Define a function $F(t) := f(t) - P_n(t) - cL(t)$ with $c = \frac{f(x) - P_n(x)}{L(x)}$

If differentiate F for $n+1$ times get $F^{(n+1)}(t) = f^{(n+1)}(t) - P_n^{(n+1)}(t) - cL^{(n+1)}(t) = f^{(n+1)}(t) - c(n+1)!$

Since $F(x_i) = 0$ for $i = 0, \dots, n$ and $F(x) = 0 \Rightarrow F$ must have at least $n+1$ distinct roots

$\Rightarrow F^{(n+1)}$ has at least 1 root, let it be δ .

$$\Rightarrow F^{(n+1)}(\delta) = 0 \Rightarrow c = \frac{1}{(n+1)!} f^{(n+1)}(\delta)$$

$$\Rightarrow f(x) - P_n(x) = \frac{1}{(n+1)!} f^{(n+1)}(\delta) \cdot L(x)$$

$$\Rightarrow R(x) := f(x) - P_n(x) = \frac{L(x)}{(n+1)!} f^{(n+1)}(\delta)$$

- (b) Consider $f(x) = x^4$ on $[0, 1]$ with $n = 2$.

- Find an upper bound for $|f'''|$ on the interval $[0, 1]$.
- Find an upper bound for $L(x)$ on the interval $[0, 1]$.
- Find an upper bound for $R(x)$,

$$(i): f'''(x) = f^{(3)}(x) = (4x^3)'' = (12x^2)' = 24x$$

$$\text{monotonically increasing} \Rightarrow f'''(0) = 0 \leq f'''(x) \leq f'''(1) = 24$$

$$\Rightarrow \text{upper bound: } 24$$

$$(ii) L(x) = \prod_{i=0}^2 (x - x_i) = (x - x_0)(x - x_1)(x - x_2)$$

$$L(x) \leq |L(x)| = |(x - x_0)(x - x_1)(x - x_2)| \leq |x - x_0| |x - x_1| |x - x_2|$$

$$\leq \max(x_0, 1 - x_0) \max(x_1, 1 - x_1) \max(x_2, 1 - x_2)$$

which is the upper bound

$$(iii) R(x) = \frac{L(x)}{3!} f^{(3)}(\delta) \quad \text{and} \quad \therefore f'''(x) < 24$$

$$\Rightarrow R(x) \leq |R(x)| \leq \frac{24}{6} |L(x)| = 4 |L(x)|$$

$$\Rightarrow \text{upper bound: } 4 \max(x_0, 1 - x_0) \max(x_1, 1 - x_1) \max(x_2, 1 - x_2)$$

Problem 2 (20 points)

Consider a least square problem $Ax = b$, where $A \in \mathbb{R}^{m \times n}$ is a tall matrix with $m > n$.

- (a) In the lecture, we discussed using the Normal Equations of Gauss, $A^T A x = A^T b$, to find the solution of the least square problem. Explain one condition where the Normal Equations of Gauss will fail.
- (b) Suppose we already have the eigendecomposition $A^T A = Q \Lambda Q^T$, where Q is orthogonal and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ contains the eigenvalues of $A^T A$ in the diagonal entries. Write down a method, with explanation, to compute the SVD decomposition of A .

- (c) Consider $A = \begin{pmatrix} 1 & 0 \\ 0 & 4 \\ 0 & 0 \end{pmatrix}$ and $b = \begin{pmatrix} 3 \\ 8 \\ 7 \end{pmatrix}$. Find the least squares solution using the pseudoinverse A^+ .

(a)

When $\text{rank}(A) \neq n$, the $A^T A$ is invertible, is not able to find the inverse of $A^T A$, the method will fail. Also, under the condition where $\kappa(A)$ is large so $\kappa(A^T A)$ is extremely large, the method could also fail due to large computation error accumulate in the calculation of solving linear systems.

(b)

Since $A^T A = Q \Lambda Q^T$ and to find $A = U \Sigma V^T$

$$\Rightarrow A^T A = (U \Sigma V^T)^T (U \Sigma V^T) = V^T \Sigma^T U^T U \Sigma V^T = V \Sigma^T \Sigma V$$

$$\text{Since } \Sigma = \text{diag}(\sigma_1, \dots, \sigma_n), \Rightarrow \Sigma^T \Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_n^2)$$

$$\Rightarrow \sigma_i = \sqrt{\lambda_i} \text{ to find } \Sigma$$

$$V = Q,$$

$$u_i = \frac{1}{\sigma_i} A v_i \text{ where } u_i, v_i \text{ are } i^{\text{th}} \text{ columns of } U, V \text{ to find } U.$$

(c)

$$\text{To find SVD of } A: A^T A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 16 & 0 \\ 0 & 0 & 0 \end{pmatrix} \Rightarrow \lambda_1 = 1, v_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}; \lambda_2 = 16, v_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}; \lambda_3 = 0, v_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

$$\Rightarrow A = U \Sigma V^T = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 4 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$\Rightarrow A^+ = U \Sigma^+ V^T = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{4} & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{4} & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

By pseudo inverse method:

$$\begin{aligned} \text{Full solution set } S &= \{x: x = A^+ b + (I - A^+ A)w \text{ for } \forall w\} \\ &= \{x: x = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{4} & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 3 \\ 8 \\ 7 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}\} \end{aligned}$$

$$\Rightarrow \text{solution: } x = \begin{pmatrix} 3 \\ 2 \\ 0 \end{pmatrix}$$

Problem 3 (10 points)

Let A be a diagonalizable matrix with eigenvalues $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n| > 0$ and normalized eigenvectors $v_1, \dots, v_n$. Also, consider an initial vector $x_0 = \sum_j a_j v_j$ such that $a_n \neq 0$.

(a) Write down an iterative formula for $\{x_k\}$ such that the sequence approach the direction of v_n , which corresponds to the smallest eigenvalue $|\lambda_n|$. You should avoid the direct computation of A^{-1} .

(b) Prove the convergence of your iterative formula in part (a).

(a) inverse power iteration but use linear system solution instead of directly computing A^{-1} for each iteration.

①. solve the linear system of $Ay_k = x_k$ to get y_k . (using methods in previous lectures)

②. update $x_{k+1} = \frac{y_k}{\|y_k\|}$

when converge, x_k is the normalized targeted v_n

(b). Let $x_0 = \sum_{j=1}^n a_j v_j$ and $a_n \neq 0$ where v_j are eigenvectors of A

In theory is the same with calculating A^{-1}

$$\text{since } A^{-1}v_j = \frac{1}{\lambda_j}v_j \Rightarrow (A^{-1})^k x_0 = \sum_{j=1}^n a_j \left(\frac{1}{\lambda_j}\right)^k v_j = \left(\frac{1}{\lambda_n}\right)^k \left[a_n v_n + \sum_{j < n} a_j \left(\frac{\lambda_n}{\lambda_j}\right)^k v_j \right]$$

$$\text{since } |\lambda_1| > |\lambda_2| > \dots > |\lambda_n| > 0 \Rightarrow \left|\frac{1}{\lambda_n}\right| > \left|\frac{1}{\lambda_{n-1}}\right| > \dots > \left|\frac{1}{\lambda_1}\right|$$

$\Rightarrow$ the result got from above algorithm: $\left(\frac{1}{\lambda_n}\right)^k$ the largest

$$\Rightarrow \left|\frac{\lambda_n}{\lambda_j}\right| < 1 \text{ when } j < n \Rightarrow \left(\frac{\lambda_n}{\lambda_j}\right)^k \rightarrow 0 \text{ when } k \rightarrow \infty$$

$$\Rightarrow A^k x_0 \rightarrow \left(\frac{1}{\lambda_n}\right)^k a_n v_n$$

$\Rightarrow$ when $k \rightarrow \infty$, x_k approach direction of targeted v_n .

P4. the code is in "p4.m". Run "P4" to output the results

(a). the implementation is in function `dft-direct()`

since there is nested 2 for loops of length n , and inside each time time is constant, the cost is $O(n^2)$

(b). the function is implemented in `triginterp-eval()`

(c). max error is 3.2244×10^{-15} , which is extremely small

(d). The output comparison is shown below.

n	Rel. Inf Error	Direct Time	FFT Time
16	1.6015e-15	0.0001	0.00002
64	4.5782e-15	0.0006	0.00002
256	1.5218e-14	0.0068	0.00006
1024	4.0981e-14	0.0870	0.00007
4096	1.7508e-13	1.4972	0.00009

The error between the two methods are small, neglectable

when n is small, the time is both fast

when n increases, in direct FFT's time increase rapidly, but FFT time increases very slowly and FFT gets much quicker than direct FFT

P5. the code is in "P5.m". Run "P5" to output the results

(a).

$$A = \begin{pmatrix} 1 & \ln 8 \\ 1 & \ln 6 \\ 1 & \ln 8 \\ 1 & \ln 13 \\ 1 & \ln 13 \end{pmatrix}, \quad b = \begin{pmatrix} 91 \\ 98 \\ 103 \\ 110 \\ 112 \end{pmatrix} \quad \text{Size of } A: 5 \times 2$$

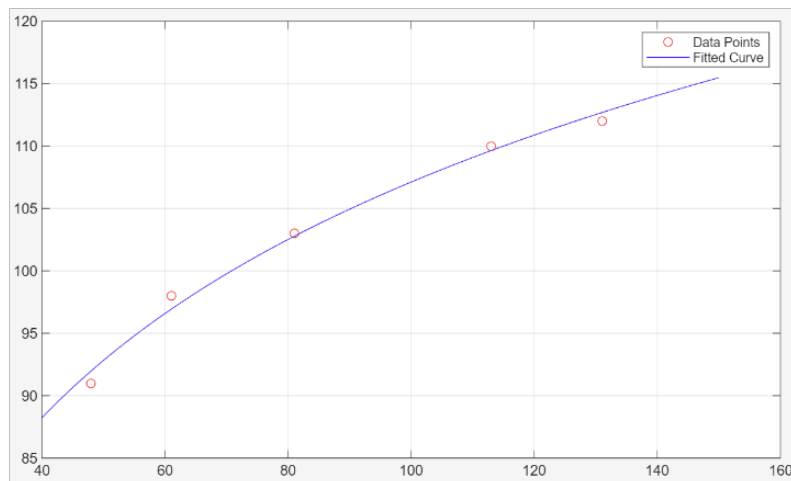
(b) the output:

Beta_0: 12.1538, Beta_1: 20.6210
Singular values: 10.1133, 0.1845

(c): the code is between line 17 and 28.

$$\|r\|_2 = 1.6658 \quad \|r\|_2 / \|b\|_2 = 0.0072$$

the curve,



(d). code is between line 30 and 37. the output:

Condition Number: 54.8206
Relative error in Beta: 2.6826e-02

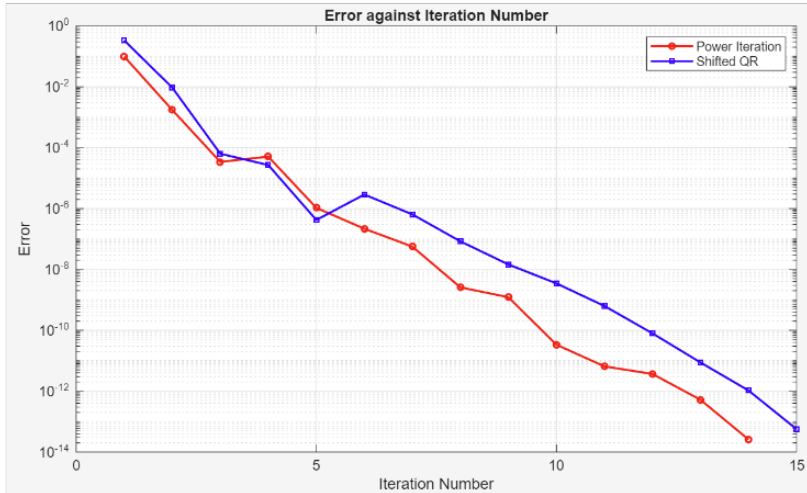
P6: the code is in P6.m. Run "P6.m" to output the results

(a) the two function are implemented in myShiftedQR() and myPowerIteration() respectively
The verification result is:

```
MATLAB eig() dominant eigenvalue: 1.000000000000002
myShiftedQR dominant eigenvalue: 1.000000000000000
myPowerIteration dominant eigenvalue: 1.000000000000099
```

which means both methods get results close to Matlab implementation

(b) the plot is shown below. the error is $\text{abs}(\text{predicted_lambda} - 1)$.



(c) The shift helps convergence. It accelerates the convergence speed

The off-diagonal values in the formula vanishes with the formula. $(E_k)_{(ij)} = (ij) \left(\frac{\lambda_i}{\lambda_j} \right)^k$,

so the speed is dependent on the size of $\frac{\lambda_i}{\lambda_j}$

By applying the shift, both the nominator and denominator is decreased, so the quotient gets smaller so after each iteration, the off-diagonal values vanish faster, which accelerates the convergence.

For the web ranking task, power iteration could be faster. Firstly, power iteration is faster. As shown in the plot it typically uses less iterations to converge, and the computation in each iteration is less than QR method. So it would be faster.

So when the page number grows, the time of shifted QR could grow and become much larger than power iteration. Also, power iteration only requires matrix vector multiplication so the computation unit operations are more effective.

Added that for the current task, only the eigenvector ~~with~~ with the largest value is needed, power iteration is a more suitable method.